AI Usage Log: Sprint 2

SOEN 341: CareerConnect
Lamees Ibeid, Backend Team

AI Tool Used: Claude (Anthropic).

My Sprint 2 Work: Profile endpoints and the logged-in user helper (LA-1), resume endpoints (LA-2), the apply-for-a-job endpoint (LA-3), and saving the optional cover note on applications.

Task 1: Profile Endpoints and the Logged-In User Helper (LA-1)

Purpose of AI Use: Code generation and learning (authentication with tokens, FastAPI dependencies, create-or-update logic)

Chat Link or Prompt/Response: Prompt (paraphrased): “Help me build LA-1 (the profile endpoints) one file at a time, and explain each file, because token authentication is new to me.” “Walk me through testing the endpoints in /docs and opening the pull request.” Response: The AI checked the database tables Kaila created, the frontend profile page Krish built and the API cheat sheet, then wrote four files with explanations: a helper that finds the logged-in user, the profile schemas, the profile router with `GET /profile/me` and `PUT /profile/me`, and nine tests. It also explained the two lines to add to main.py and gave me a commit message and pull request text.

AI-Suggested Content: When someone logs in, Youssef's code gives them a token (a JWT): a signed piece of text that contains their user id in a field called `sub`. The frontend sends it back on every request in the header `Authorization: Bearer <token>`. The AI explained that the helper `get_current_user` reads that header, checks the signature with our secret key (so nobody can fake or edit a token), checks it has not expired, takes out the user id and loads the user from the database. If anything is wrong it returns 401 "Not logged in." It explained that this is written once and shared, because every endpoint that needs a logged-in user (mine and Youssef's) uses it.

The AI explained FastAPI's `Depends(...)`: instead of every endpoint repeating the token code, an endpoint just asks for `current_user: User = Depends(get_current_user)` and FastAPI runs the helper first. If the helper fails, the endpoint never runs. The same idea is used for the database session with `Depends(get_db)`. Using `HTTPBearer` also adds the Authorize button to the /docs page, so I can paste a token there and test logged-in endpoints.

The endpoint is `/profile/me`, not `/profile/5`. If the user id came from the address, anyone could change the number and read someone else's profile. Taking the id from the token means

each user can only ever reach their own profile.

`PUT /profile/me` first looks for the user's profile. If there is none, it creates a new row; if there is one, it updates it. The AI explained the loop that copies every field from the request into the row using `model_dump()` and `setattr`, and that `db.commit()` is what actually saves the change (forgetting it is a common mistake).

`ProfileIn` holds the rules from my guide: the full name is required and cannot be only spaces, the summary is at most 2000 characters and there are at most 30 skills. `ProfileOut` turns empty database values into "" and [] because Krish's page expects text and lists, never null.

Validation: The AI ran the whole test suite before giving me the code (26 passed: the 17 existing tests and 9 new ones) and tried the endpoints on a running server. The new tests check: GET before saving gives 404, saving and reading back returns the same data, a second save updates instead of creating a duplicate, a blank name is rejected, more than 30 skills and a summary over 2000 characters are rejected, a second user cannot see the first user's profile, and requests without a valid token get 401. I compared the field names with Krish's frontend profile type and the API cheat sheet. I then ran pytest on my laptop (26 passed) and tested by hand in /docs: I registered, logged in, pasted the token with the Authorize button, and checked that GET gave 404, PUT gave 200 and GET then returned my profile. My first PUT returned 401 because I had not authorized yet, which confirmed the helper rejects requests without a token.

Decision: Modified before use. The AI suggested naming the branch after the task ID (feature/LA-1-profile-endpoints), as in the team guide; I chose feature/profile-endpoints instead. I also asked for a much shorter pull request description. The code itself was accepted, with one difference from the cheat sheet: invalid input returns 400 instead of 422, to stay consistent with the error rule we set in Sprint 1 that the frontend already relies on.

Reflection: This was the first time I worked with authentication on the backend, and I now understand how the token connects login to everything else: login creates it, the frontend stores and sends it, and the helper turns it back into a user. The point that clicked the most was why the address says /me instead of a user id, because it is a security decision and not just a naming choice. I also learned what Depends does, which I had seen in the Sprint 1 code without understanding it.

Responsible Person: Lamees Ibeid

Task 2: Resume Endpoints (LA-2)

Purpose of AI Use: Code generation and learning (file uploads, path parameters, reusing teammates' functions)

Chat Link or Prompt/Response: Prompt (paraphrased): “Help me build LA-2 (the resume endpoints) and their tests, one file at a time, and check what changed on GitHub since LA-1 first.” Response: The AI checked GitHub first and found that my LA-1 pull request was not merged yet, so it had me create the new branch from my LA-1 branch (which contains the logged-in user helper) instead of from main. It also found that Youssef had written a second version of the same helper on his branches and suggested a message to coordinate. It then gave me three files for LA-2 (a resume schema, a router with five endpoints and eleven tests) and explained each setup step.

AI-Suggested Content: The AI explained why the branch had to start from my LA-1 branch (the resume endpoints need `get_current_user`, which only exists there until LA-1 is merged), and why `python-multipart` had to be both installed and added to requirements.txt: installing it only fixes my laptop, while requirements.txt is what teammates and CI install from.

An endpoint receives an upload with `file: UploadFile = File(...)` and reads its bytes with `await file.read()`. The AI explained that `async`/`await` lets the server keep working while the file arrives.

One shared function checks every upload and replacement: the type must be PDF or Word, the extension must match (because a file name alone can lie), the file cannot be empty and it must be at most 5 MB. Otherwise the endpoint returns 422 with a plain message. The limits come from Kaila's rules file, so there is only one place to change them.

In `/resumes/3/download`, the 3 is a path parameter. The AI explained why this is safe even though the profile endpoint avoided ids in the address: Kaila's `get_resume` only finds a resume that also belongs to the logged-in user, and someone else's resume returns the same 404 as a missing one, so nobody can discover which ids exist.

The download returns the raw bytes with a `Content-Disposition: attachment` header so the browser saves the file, and a successful delete returns 204 (done, nothing to send back). The schema `ResumeOut` never includes the file itself, so listing resumes stays small.

Validation: The AI ran the tests and tried an upload and download on a running server before giving me the code. On my laptop, pytest showed 37 passed (26 existing and 11 new). The new tests check uploading a PDF and a Word file, listing, downloading the exact same bytes, replacing, deleting, rejecting an .exe, an empty file and a file over 5 MB, blocking another user

from every action on my resume, and requiring login. I checked that the response fields match the resume type in Krish's frontend code.

Decision: Accepted.

Reflection: File uploads were completely new to me. I now understand how a file travels from the browser to the database as bytes, why the server has to check the type and size instead of trusting the file name, and why returning 404 instead of 403 protects other users' privacy. I also learned to reuse a teammate's functions instead of writing new database code, which kept my endpoints short.

Responsible Person: Lamees Ibeid

Task 3: Apply-for-a-Job Endpoint (LA-3)

Purpose of AI Use: Code generation and learning (business rules, permissions, handling duplicate requests)

Chat Link or Prompt/Response: Prompt (paraphrased): “Walk me through LA-3, the apply-for-a-job endpoint, and its tests.” “Write the messages I should send to Sayf, Youssef and Krish about my endpoints.” Response: The AI checked Sayf's applications table and Kaila's job and resume functions, then gave me four files: the five official status words as constants, the request and response schemas, the endpoint with four rules, and seven tests. It also pointed out two problems it found in the existing code: the applications table has no column for the optional cover note in the cheat sheet, and the table's automatic date uses a PostgreSQL-only function that does not work on my local SQLite database. Finally, it had me update three profile tests so they accept either 400 or 422, because Youssef's unmerged branch changes the error code.

AI-Suggested Content: The endpoint checks four rules in order, and the first one that fails stops the request with its own error: only job seekers can apply (403), the job must exist (404) and be open (422), the resume must belong to the user (404), and the user cannot apply twice to the same job (409). The AI explained the difference between 401 (we do not know who you are) and 403 (we know who you are, but you are not allowed).

The five statuses (Applied, Under Review, Interview, Offered, Rejected) are written once in `App/application_statuses.py` and imported everywhere, so a typo like "applied" cannot happen. Youssef's status endpoints can reuse the same file.

Two requests arriving at the same moment could both pass the duplicate check. Sayf's table has a unique rule on user and job, so the second save fails with an `IntegrityError`; the endpoint catches it, undoes the change with `db.rollback()` and returns the same 409 instead of crashing. This is the same technique I used for duplicate emails in Sprint 1 registration.

The endpoint sets `applied_at` itself because the table's default only works on PostgreSQL. Without it, applying would crash on my local database.

The test database starts empty, so a helper adds a job directly through the same temporary database the tests use. The tests reuse the upload helper from my resume tests and the second-user helper from my profile tests.

Validation: The AI ran the tests and tried uploading a resume, applying, and applying again on a running server with a database built from the team's migrations, before giving me the code. On my laptop, pytest showed 44 passed (37 existing and 7 new). The new tests check a successful application with status "Applied" and the message "Application submitted", applying twice, a missing job, a closed job, using another user's resume, a recruiter trying to apply, and applying without logging in.

Decision: Modified before use. The optional cover note from the cheat sheet was left out because the applications table has no column to store it, and the profile tests were changed to accept 400 or 422.

Reflection: This task showed me how a real feature is mostly rules. Writing them as a list of guards, each with its own status code and message, made the code easy to read and test. The double-click problem was new to me: I did not know two requests could both pass a check at the same moment, and I learned that the database rule is the real protection while the earlier check gives a friendlier message. I also learned that code has to work on both the local and the shared database, which is why the date is set in code.

Responsible Person: Lamees Ibeid

Task 4: Saving the Optional Cover Note

Purpose of AI Use: Code generation, debugging and learning (database migrations)

Chat Link or Prompt/Response: Prompt (paraphrased): “Sayf asked me to accept an optional cover note (up to 1000 characters) and save it when an application is created. Tell me exactly what to fix.” “Help me understand the errors I am getting when I run pytest and alembic.” Response: The AI checked main and found that the applications table did not actually have a cover_note column, so the fix needed a database migration as well as code changes. It gave me five small changes (the table model, a new migration file, the schema, the endpoint and three tests), a message for Sayf announcing the migration, and the pull request text. It then helped me read and fix three errors.

AI-Suggested Content: The AI explained that changing the Python model only changes the description of the table, while the real databases (my local one and the team's Neon database)

only get the new column when a migration runs. In the migration file, revision is the migration's own id and down_revision is the one right before it, which is how Alembic knows the order: initial tables, then the applications table, then the cover note. upgrade() adds the column and downgrade() removes it.

In the schema, `cover_note: str | None = Field(default=None, max_length=1000)` makes the field optional (it can be text or nothing) and rejects anything over 1000 characters. In the endpoint, one line copies it into the new database row. Because the endpoint's reply does not include the cover note, the tests read it directly from the test database to prove it was really saved.

When pytest failed with an IndentationError, the AI explained that Python requires every line in a block to start with the same number of spaces, and that VS Code adds its own indentation when pasting on an already indented line. When alembic heads still showed the old migration, it had me list the versions folder and search the project, which showed I had created the file in the alembic folder instead of alembic/versions. After moving it, a SyntaxError showed a leftover quote from removing the comments, and the AI showed me how to read only the last lines of a long error to find the file and line.

Validation: After the fixes, alembic heads showed the new migration, alembic upgrade head added the column to my local database, and pytest showed 53 passed, including three new tests: the cover note is saved, applying without one still works, and a note over 1000 characters is rejected. Before giving me the migration, the AI also tested it on a new database and on an existing one, including undoing it.

Decision: Modified before use. I used a shorter version of the migration file without the optional comments, and I told Sayf that the column did not exist yet and that someone with the Neon link must run the migration after merging.

Reflection: This was my first database migration, and I now understand why changing the model alone is not enough. The errors taught me as much as the code: indentation matters in Python, a migration in the wrong folder is simply ignored, and the last lines of a long error usually say exactly what is wrong. I also learned to check the actual code instead of assuming, since the request said the column already existed and it did not.

Responsible Person: Lamees Ibeid